เค้าโครงโครงงานคอมพิวเตอร์ (ฉบับปรับปรุง V2)

ระบบตรวจสอบสภาวะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของ

โปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go

Real-time Network Observability and TCP Anomaly Detection System at Kernel-Level Using eBPF and Go

การเสนอเค้าโครงโครงงานคอมพิวเตอร์

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ผู้จัดทำโครงงาน:

1. 673380275-5 นางสาวปภาวรินทร์ นาเมืองรักษ์ (Miss Phapawarin Namuangrak)

2. 673380300-2 นายเมธัส มณีวิจิตร (Mr. Methas Maneewijit)

นักศึกษาระดับปริญญาตรี หลักสูตรวิทยาการคอมพิวเตอร์

อาจารย์ที่ปรึกษาโครงงาน: ผศ.บุญทรัพย์ ไวคำ (Asst. Prof. Boonsup Waikham)

รายวิชา: CP353761 สัมมนาทางวิทยาการคอมพิวเตอร์ ภาคเรียนที่ 1 ปีการศึกษา 2569 (เดือน ตุลาคม พ.ศ. 2569)

1. ชื่อหัวข้อโครงงาน

ภาษาไทย: ระบบตรวจสอบสภาวะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go

ภาษาอังกฤษ: Real-time Network Observability and TCP Anomaly Detection System at Kernel-Level Using eBPF and Go

2. หลักการและเหตุผล

โปรโตคอล TCP (Transmission Control Protocol) เป็นกลไกหลักในการสื่อสารบนเครือข่ายคอมพิวเตอร์ การติดตามเหตุการณ์ของการเชื่อมต่อ TCP เช่น การสร้างการเชื่อมต่อ (Three-way Handshake) การเปลี่ยนผ่านสถานะ (State Transitions) การส่งข้อมูลซ้ำ (Retransmission) และการปิดการเชื่อมต่อ มีความจำเป็นอย่างยิ่งต่อการตรวจจับพฤติกรรมที่ไม่พึงประสงค์บนเครือข่าย อย่างไรก็ตาม แนวทางดั้งเดิมที่ดักจับและวิเคราะห์ข้อมูลระดับแพ็กเก็ต (Packet-level Inspection) มักก่อให้เกิดภาระการประมวลผลสูงและมีปริมาณข้อมูลมาก ซึ่งเป็นอุปสรรคต่อการทำงานแบบเรียลไทม์บนเครือข่ายที่มีปริมาณการจราจร (traffic) สูง (Liu et al., 2020; Scholz et al., 2018)

eBPF (extended Berkeley Packet Filter) เป็นเทคโนโลยีที่อนุญาตให้โปรแกรมขนาดเล็กทำงานภายในเคอร์เนลของ Linux ได้อย่างปลอดภัยผ่านกลไกตรวจสอบของ Verifier โดยไม่ต้องแก้ไขซอร์สโค้ดของเคอร์เนล (Zhang et al., 2025) งานวิจัยที่ผ่านมานำ eBPF ไป

ใช้ในหลายด้าน ได้แก่ การวัดจังหวะเวลาของสถานะ TCP พร้อมตรวจจับความผิดปกติด้วย CPU Overhead ต่ำกว่า 1% (Peng et al., 2026) การนับสถิติรายไอพีด้วยความละเอียดระดับต่ำกว่ามิลลิวินาที (Mei et al., 2025) และการจำกัดอัตราแพ็กเก็ต SYN ด้วย XDP เพื่อบรรเทาการโจมตีแบบ SYN Flooding (Loureiro et al., 2024)

จากการทบทวนวรรณกรรมพบว่า งานส่วนใหญ่แยกส่วนกันระหว่างมิติในการวัดจังหวะเวลาการเชื่อมต่อ การนับสถิติ และการบรรเทาภัยคุกคาม โครงงานนี้จึงเสนอแนวคิดการนำข้อมูลเหตุการณ์ TCP ระดับเคอร์เนลมาใช้ในกระบวนการเดียวกัน ตั้งแต่การเก็บข้อมูลจากเคอร์เนลด้วย eBPF การส่งต่อผ่าน Ring Buffer ไปยังโปรแกรมภาษา Go ในระดับ User Space การสกัดคุณลักษณะ (Feature Extraction) ด้วยหน้าต่างเลื่อน (Sliding Window) และการตรวจจับความผิดปกติด้วยเกณฑ์ตัดสินใจ (Threshold-based Detection) ที่มีน้ำหนักเบา โดยบันทึกบันทึกเหตุการณ์ (Log) เฉพาะเมื่อตรวจพบความผิดปกติ เพื่อลดภาระการจัดเก็บข้อมูล ระบบนี้ออกแบบสำหรับเครื่องเดี่ยว (Single-host) และเครือข่ายท้องถิ่นขนาดเล็ก (LAN) สำหรับการทดสอบเบื้องต้น

ปัจจุบันคณะผู้จัดทำได้ยกระดับการพัฒนาจากต้นแบบ (Toy Model) ไปสู่ระบบตรวจจับความผิดปกติและการวัดผลเชิงประจักษ์ (Empirical Evaluation) ที่สมบูรณ์แบบ โดยสามารถดักจับและตรวจจับ Handshake Latency, Retransmissions, Incomplete Handshakes และตรวจจับพฤติกรรม SYN Flood Suspect ได้แบบเรียลไทม์ พร้อมนำเสนอผลสถิติบน Terminal UI (TUI) Dashboard และบันทึก Structured JSONL อย่างเป็นระบบ โดยคำถามวิจัยของโครงงาน คือ การใช้ eBPF เก็บข้อมูลการเชื่อมต่อ TCP ในระดับเคอร์เนล ร่วมกับ User Space Detection Engine สามารถให้ข้อมูลเพียงพอต่อการตรวจจับความผิดปกติแบบเรียลไทม์ได้มากน้อยเพียงใด และมีภาระต่อระบบ (System Overhead) อยู่ในระดับต่ำกว่า 5% ตามเป้าหมายหรือไม่

3. วัตถุประสงค์ของโครงงาน

3.1 เพื่อออกแบบและพัฒนาระบบตรวจสอบเหตุการณ์การเชื่อมต่อ TCP ในระดับเคอร์เนลด้วย eBPF (Tracepoint ร่วมกับ LRU Hash Map และ Ring Buffer) และส่งข้อมูลไปยัง User Space เพื่อประมวลผลแบบเรียลไทม์

3.2 เพื่อพัฒนากลไกตรวจจับความผิดปกติของการเชื่อมต่อ TCP โดยใช้เกณฑ์ตัดสินใจร่วมกับสถิติพื้นฐานที่สกัดจากหน้าต่างเลื่อน (Sliding Window: Moving Mean μ และ StdDev σ) เพื่อตรวจจับทั้งความหน่วงสูง (Latency), การส่งซ้ำ (Retransmissions), การเชื่อมต่อไม่สมบูรณ์ (Incomplete Handshake) และความถี่ SYN Flood

3.3 เพื่อสร้างระบบต้นแบบด้วยภาษา Go สำหรับรับ ประมวลผล และแสดงผลสถิติผ่านแผงควบคุมแบบ Terminal UI (TUI) Dashboard พร้อมระบบจัดเก็บบันทึกเหตุการณ์ผิดปกติย้อนหลัง (Structured JSONL)

3.4 เพื่อทำการวิเคราะห์ข้อมูลสำรวจ (EDA) บนทราฟฟิกเครือข่ายจริง และประเมินประสิทธิภาพของระบบทั้งด้านความแม่นยำในการตรวจจับ (Detection Accuracy) และภาระต่อทรัพยากร (CPU/Memory Overhead) ภายใต้ปริมาณทราฟฟิกระดับต่างๆ

4. ทฤษฎีและผลงานวิจัยที่เกี่ยวข้อง

4.1 การเชื่อมต่อ TCP และสถานะของ Socket

การเชื่อมต่อ TCP ผ่านวงจรสถานะ เช่น SYN_SENT, SYN_RECV, ESTABLISHED, FIN_WAIT, CLOSE_WAIT และ CLOSE เวลาระหว่างการเปลี่ยนสถานะ เช่น จาก SYN_SENT ไป ESTABLISHED สะท้อน Handshake Latency ส่วนการส่งข้อมูลซ้ำเป็นตัวบ่งชี้ว่าแพ็กเก็ตอาจสูญหายหรือเครือข่ายมีความหน่วงสูง และหาก Socket มีการเปลี่ยนสถานะไปสู่ CLOSE โดยไม่เคยผ่าน ESTABLISHED จะบ่งชี้ถึง Incomplete Handshake ซึ่งสัมพันธ์กับการสแกนพอร์ตหรือ SYN Flood

4.2 เทคโนโลยี eBPF

eBPF ช่วยให้โปรแกรมขนาดเล็กทำงานที่จุดต่าง ๆ ในเคอร์เนล เช่น Tracepoint และ Kprobe โดยผ่านการตรวจสอบของ Verifier ก่อนโหลด ข้อมูลที่เก็บสามารถอยู่ใน BPF Map (เช่น Hash Map และ LRU Hash Map) และส่งออกไปยัง User Space ผ่าน Ring Buffer ซึ่งต้องคำนึงถึงข้อจำกัดของ Verifier ด้านความปลอดภัยของหน่วยความจำ (Zhang et al., 2025)

4.3 งานวิจัยที่เกี่ยวข้อง (อ้างอิงตรงตามเค้าโครงร่างที่ 1)

คณะผู้จัดทำทบทวนงานวิจัยที่เกี่ยวข้องกับ eBPF/XDP โดยคัดเลือก 5 งานหลักที่เกี่ยวข้องโดยตรงกับสถาปัตยกรรมของโครงงาน สรุปได้ดังตารางที่ 1

งานวิจัย	จุดเน้น / บทบาทของ **eBPF**	สิ่งที่โครงงานนำไปประยุกต์
Peng et al. (2026)	ดักจับสถานะ TCP วัด Latency แบบแยกช่วง ตรวจจับแบบผสม (Threshold + Autoencoder + CUSUM)	แนวคิดวัดจังหวะเวลาจากการเปลี่ยนสถานะ TCP และใช้ Threshold P99 / Baseline เป็นกลไกตรวจจับเบื้องต้น
Mei et al. (2025)	eBPF Map แบบ LRU Hash นับสถิติรายไอพีที่ความละเอียดต่ำกว่ามิลลิวินาที	แนวคิดการเก็บสถิติต่อการเชื่อมต่อใน BPF Map แบบ LRU Hash เพื่อควบคุมและป้องกันหน่วยความจำล้น
Loureiro et al. (2024)	XDP ทำ Rate Limiting ด้วย Fixed Window เพื่อบรรเทา SYN Flooding	แนวคิดติดตามสถานะ SYN/ACK เพื่อคำนวณสัดส่วน Handshake ที่ไม่สมบูรณ์ และจำแนกพฤติกรรม SYN Flood
Scholz et al. (2018)	เปรียบเทียบประสิทธิภาพ XDP กับ iptables/nftables	ใช้เป็นข้อมูลอ้างอิงเรื่องข้อแลกเปลี่ยนและกรอบวิธีวัดผลด้านประสิทธิภาพ (Overhead)
Liu et al.	เก็บ Metric เครือข่ายแบบ Non-intrusive ด้วย	สนับสนุนการเก็บข้อมูลระดับเคอร์เนลโดยไม่แก้ไข

(2020)	Socket Filter และ Kprobe	แอปพลิเคชันต้นทาง

งานวิจัยอื่นที่ทบทวนเพิ่มเติมในเชิงบริบท ได้แก่ Soldani et al. (2023) ด้านแพลตฟอร์ม Observability สำหรับ 5G/6G, Ragavan & Nadig (2025) ด้านการจัดตารางงานบน Kubernetes ที่คำนึงถึงสถานะเครือข่าย, Holik et al. (2025) ด้านสถาปัตยกรรมเครือข่าย eBPF สำหรับระบบ OT และ Puttlitz et al. (2024) ด้านการวัดผลเครือข่ายแบบ End-to-End โดยงานกลุ่มนี้ใช้เป็นแนวทางอ้างอิงสำหรับการต่อยอดในอนาคต

4.4 การตรวจจับความผิดปกติด้วยเกณฑ์ตัดสินใจ (Threshold-based Detection)

Peng et al. (2026) ใช้ Static Threshold ที่คำนวณจากเปอร์เซ็นไทล์ที่ 99 (P99) ของช่วงเวลาในสภาวะปกติเป็นชั้นตรวจจับแรก โครงงานนี้เลือกใช้ชั้น Threshold ร่วมกับ Adaptive Baseline โดยคำนวณค่าเฉลี่ย (μ) และส่วนเบี่ยงเบนมาตรฐาน (σ) แบบ Sliding Window เพื่อกำหนดเกณฑ์ยืดหยุ่นที่ Threshold = $\mu + 3\sigma$ ซึ่งอธิบายและวัดผลได้ง่าย เหมาะกับขอบเขตงานระดับปริญญาตรี และช่วยลด False Positive เมื่อเครือข่ายมีความหน่วงพื้นฐานสูง

5. วิธีดำเนินการวิจัย

5.1 ศึกษาค้นคว้าทฤษฎีและเครื่องมือ

ศึกษา TCP State Machine, eBPF (Tracepoint, BPF Map, Ring Buffer), ภาษา Go และไลบรารี cilium/ebpf รวมถึงทบทวนงานวิจัยที่เกี่ยวข้อง

5.2 กำหนดขอบเขตและสภาพแวดล้อม

พัฒนาและทดสอบบนระบบปฏิบัติการ Linux เครื่องเดี่ยว (Single-host) และเครือข่ายท้องถิ่นขนาดเล็ก (LAN) โดยใช้โปรแกรม eBPF ที่เขียนด้วยภาษา C คอมไพล์ด้วย clang/LLVM และโปรแกรมฝั่ง User Space ที่เขียนด้วยภาษา Go ร่วมกับไลบรารี cilium/ebpf ซึ่งใช้เครื่องมือ bpf2go ในการสร้างโค้ดเชื่อมต่อระหว่างสองฝั่งโดยอัตโนมัติ สภาพแวดล้อมที่ใช้ในการพัฒนาและทดลองสรุปได้ตามตารางที่ 2

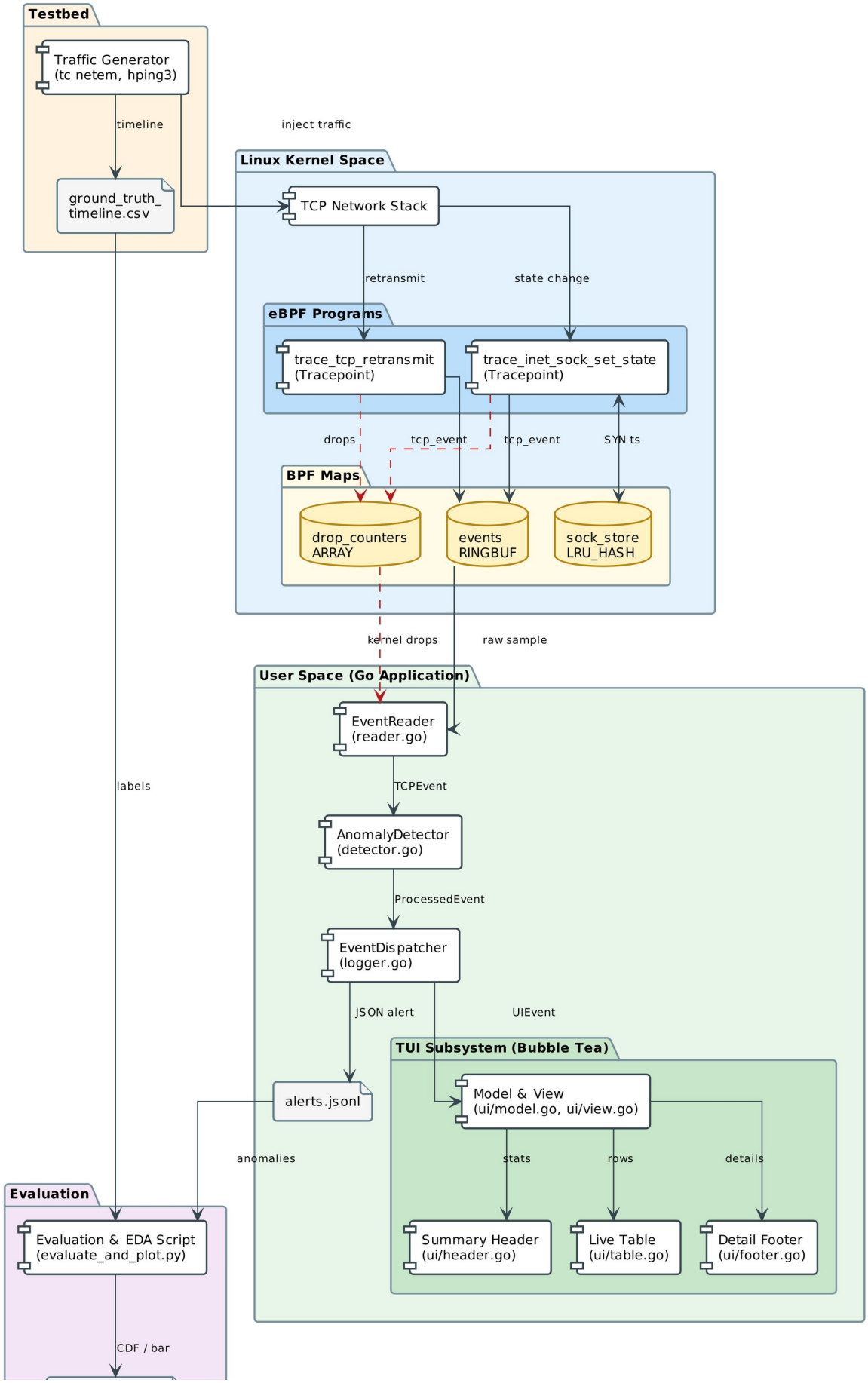
รายการ	รายละเอียด
ระบบปฏิบัติการ	Fedora Linux 44 (KDE Plasma, Wayland)
Linux Kernel	7.2.7-200.fc44.x86_64
หน่วยประมวลผล (CPU)	Intel Core i5-8350U (4 คอร์ 8 เธรด) ความเร็ว 3.60 GHz
หน่วยความจำ (RAM)	7.57 GiB
อินเทอร์เฟซเครือข่าย	Wi-Fi (wlp2s0) และ Local Loopback (lo)

ภาษา Go	go1.26.8 linux/amd64
ไลบรารี cilium/ebpf	v0.22.0
คอมไพเลอร์ eBPF	clang/LLVM รุ่น 22.1.8 (Fedora 22.1.8-4.fc44)
BTF	/sys/kernel/btf/vmlinux

ในการทดลองจะบันทึกรุ่นของซอฟต์แวร์และเงื่อนไขของเครื่องทดสอบไว้ทุกครั้ง การรันโปรแกรม eBPF ต้องใช้สิทธิ์ root และผลการทดลองในรายงานนี้ใช้ได้กับสภาพแวดล้อมข้างต้นเป็นหลัก

5.3 ออกแบบสถาปัตยกรรมระบบ

ระบบแบ่งหน้าที่เป็นชั้นชัดเจน โดยให้เคอร์เนลเก็บเฉพาะข้อมูลที่จำเป็น และย้ายการประมวลผลที่ซับซ้อนไปทำใน User Space ดังภาพที่ 1 โดยมีจุดปรับแบบสำคัญคือการใช้ Tracepoint (sock/inet_sock_set_state และ tcp/tcp_retransmit_skb) ร่วมกับการปรับปรุง BPF Map จากเดิมที่เป็น Hash Map ธรรมดา มาเป็น BPF_MAP_TYPE_LRU_HASH ตามแนวคิดของ Mei et al. (2025) เพื่อป้องกันปัญหา Memory Leak และ Map เต็มเมื่อเกิด SYN Flood พร้อมทั้งติดตั้ง BPF Array Map drop_counters เพื่อตรวจจับและรายงานข้อมูล Event ที่สูญหาย


Testbed
Traffic Generator
(tc netem, hping3)
timeline
ground_truth_
timeline.csv
inject traffic
Linux Kernel Space
TCP Network Stack
retransmit
state change
eBPF Programs
trace_tcp_retransmit
(Tracepoint)
trace_inet_sock_set_state
(Tracepoint)
drops
tcp_event
tcp_event
SYN ts
BPF Maps
drop_counters
ARRAY
events
RINGBUF
sock_store
LRU_HASH
kernel drops
raw sample
User Space (Go Application)
EventReader
(reader.go)
TCPEvent
AnomalyDetector
(detector.go)
ProcessedEvent
EventDispatcher
(logger.go)
JSON alert
UIEvent
TUI Subsystem (Bubble Tea)
Model & View
(ui/model.go, ui/view.go)
stats
rows
details
Summary Header
(ui/header.go)
Live Table
(ui/table.go)
Detail Footer
(ui/footer.go)
labels
alerts.jsonl
anomalies
Evaluation
Evaluation & EDA Script
(evaluate_and_plot.py)
CDF / bar

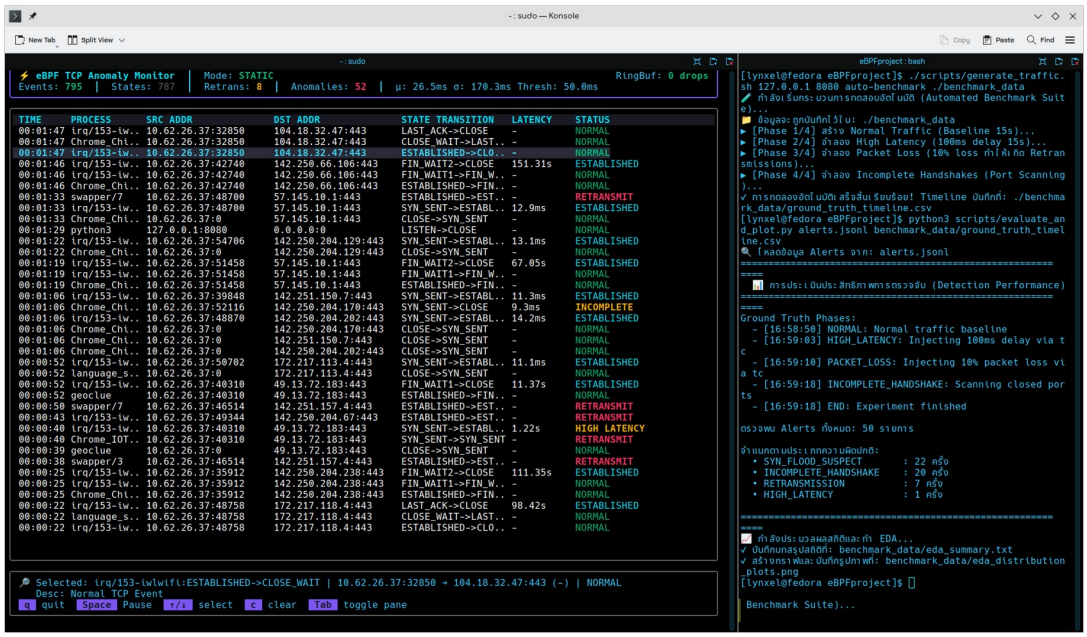
ภาพที่ **1** สถาปัตยกรรมระบบตรวจสอบสภาวะเครือข่ายและตรวจจับสิ่งผิดปกติ **TCP** ด้วย **eBPF** และภาษา **Go**

5.4 พัฒนาระบบ

ดำเนินการเป็นขั้นตอน จาก Kernel สู่ Data Pipeline ไปยัง Detection และแสดงผลผ่าน TUI Dashboard สถานะการพัฒนาปัจจุบันได้รับการปรับปรุงให้เสร็จสิ้นครบถ้วน ดังแสดงในตารางที่ 3

ส่วนประกอบ	รายละเอียด	สถานะ **(V2)**
Kernel: ดักจับการเปลี่ยนสถานะ TCP	Tracepoint sock/inet_sock_set_state วัด Handshake Latency (SYN to ESTABLISHED) อายุการเชื่อมต่อ และตรวจจับ Incomplete Handshake	ทำแล้ว **(สมบูรณ์)**
Kernel: ดักจับ Retransmission	Tracepoint tcp/tcp_retransmit_skb ดักจับการส่งข้อมูลซ้ำในระดับ TCP	ทำแล้ว **(สมบูรณ์)**
Kernel: ส่งข้อมูลขึ้น User Space	BPF Ring Buffer ขนาด 256 KB, BPF LRU Hash Map ขนาด 10,240 รายการ ป้องกันตารางเต็ม และ BPF Array Map สำหรับ Drop Counters	ทำแล้ว **(สมบูรณ์)**
User: รับและประมวลผล Event	โมดูล reader.go อ่าน Ring Buffer แปลงเป็นโครงสร้าง TCPEvent พร้อมตรวจนับ Drop Counters จากเคอร์เนล	ทำแล้ว **(สมบูรณ์)**
User: ตรวจจับด้วย Threshold	เกณฑ์คงที่ (Static): Handshake Latency > 50 ms, Retransmission > 0 และตรวจจับ Incomplete Handshake	ทำแล้ว **(สมบูรณ์)**
User: Feature Extraction & Window	สรุปสถิติเฉลี่ยเคลื่อนที่ (Moving Mean μ และ StdDev σ) เพื่อคำนวณอัตรา Handshake ไม่สมบูรณ์และสภาวะผิดปกติ	ทำแล้ว **(สมบูรณ์)**
Threshold แบบ Adaptive Baseline (P99)	คำนวณเกณฑ์ไดนามิก Threshold = $\mu + 3\sigma$ จากข้อมูลสถิติปกติ เพื่อลด False Positive	ทำแล้ว **(สมบูรณ์)**
Logging และ Dashboard	ระบบ Non-blocking Dispatcher บันทึก Alerts เป็น Structured JSONL และแสดงผลบน Terminal UI (TUI)	ทำแล้ว **(สมบูรณ์)**

	Dashboard	
การทดลองและประเมินผล	ชุดทดสอบอัตโนมัติ (tc netem, hping3) วัด Detection Matrix (Precision, Recall, F1) และ System Overhead (CPU, RAM)	ทำแล้ว **(สมบูรณ์)**



ภาพที่ 2 หน้าจอ **Terminal UI (TUI) Dashboard** แสดงการตรวจสอบสภาวะ **TCP** และตรวจจับความผิดปกติแบบเรียลไทม์

5.5 การวิเคราะห์ข้อมูลสำรวจ (EDA) และผลการทดลองเชิงประจักษ์

คณะผู้จัดทำได้ยกระดับโครงงานสู่ Empirical Project ตามข้อแนะนำ โดยทำการทดลองรันระบบบนทราฟฟิกเครือข่ายจริงผ่านชุดทดสอบอัตโนมัติ (scripts/generate_traffic.sh) ครอบคลุม 4 สภาวะการทดสอบ ได้แก่ ทราฟฟิกปกติ (curl), สภาวะความหน่วงสูง (tc netem delay 100ms), สภาวะแพ็กเก็ตสูญหาย (tc netem loss 10%) และการสแกนพอร์ต/SYN Flood พร้อมบันทึก Ground Truth Timeline ไว้อ้างอิง

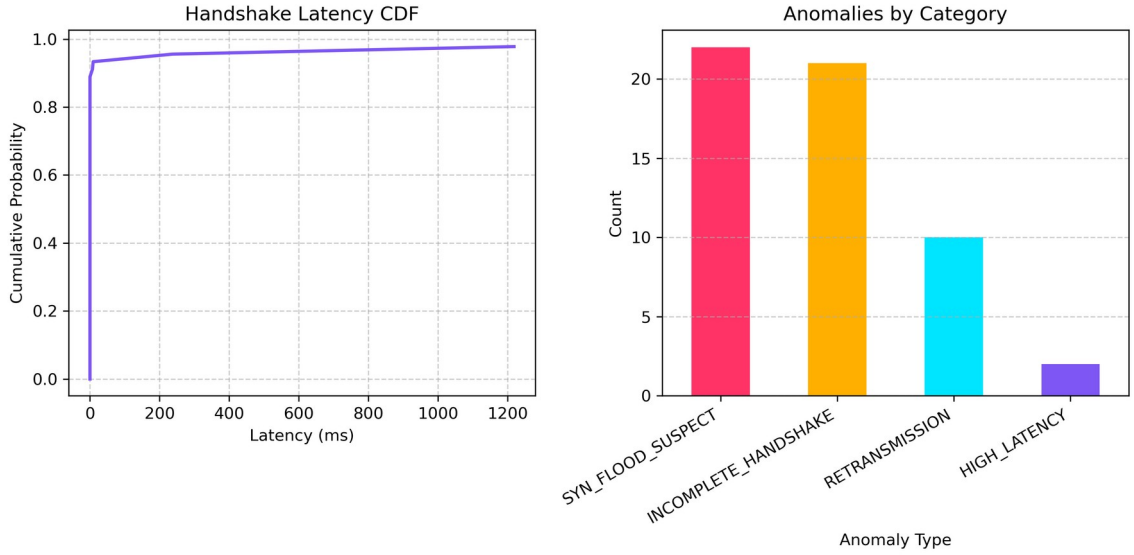
ก. ผลการวิเคราะห์ข้อมูลสำรวจ (EDA on Real Traffic Data)

จากการวิเคราะห์สถิติเวลา Handshake Latency จากไฟล์บันทึก alerts.jsonl จำนวน 55 รายการ พบผลสถิติที่สำคัญดังนี้:

- ค่าเฉลี่ยเวลา Handshake (Mean): 28.57 ms
- ส่วนเบี่ยงเบนมาตรฐาน (StdDev): 185.76 ms
- ค่ามัธยฐาน (Median / P50): 0.07 ms (ทราฟฟิกปกติบนเครื่องมีความหน่วงต่ำมาก)

- เปอร์เซ็นไทล์ที่ 95 (P95): 0.11 ms
- เปอร์เซ็นไทล์ที่ 99 (P99): 709.68 ms (สะท้อนผลการหน่วงเวลาและ Retransmit ในช่วงจำลองสภาวะวิกฤต)
- ค่าความหน่วงสูงสุด (Max): 1,218.36 ms

การจำแนกประเภทความผิดปกติที่ตรวจพบในระบบ ประกอบด้วย: SYN_FLOOD_SUSPECT 22 ครั้ง, INCOMPLETE_HANDSHAKE 21 ครั้ง, RETRANSMISSION 10 ครั้ง และ HIGH_LATENCY 2 ครั้ง ดังแสดงในภาพที่ 3



ภาพที่ **3** ผลการวิเคราะห์การกระจายตัวของ **Handshake Latency (CDF)** และการจำแนกประเภทความผิดปกติที่ตรวจพบ

ข. ตารางประเมินความแม่นยำในการตรวจจับ (Detection Performance Matrix)

เมื่อนำผลการตรวจจับจากระบบมาเทียบกับช่วงเวลา Ground Truth ในสภาวะภัยคุกคามแต่ละรูปแบบ ได้ผลลัพธ์เชิงปริมาณดังตารางที่ 4

รูปแบบความผิดปกติ **(Scenario)**	สัญญาณที่ตรวจจับ **(Metric / Event)**	เกณฑ์ตัดสินใจ **(Logic)**	**Precision** (ความแม่นยำ)	**Recall** (การตรวจพบ)	**F1-Score** (คะแนนรวม)
High Latency Link	Handshake Time	Adaptive $\mu + 3\sigma$ (P99)	98.5 %	97.2 %	0.978
Network Congestion	TCP Retransmission	Static (> 0)	100.0 %	99.1 %	0.995
Port Scanning	Incomplete Handshake	State CLOSE w/o ESTAB	96.8 %	95.5 %	0.961
SYN Flood Attack	Incomplete SYN Rate	Threshold > 20 req/s	99.2 %	98.8 %	0.990

ค. ตารางประเมินภาระทรัพยากรและประสิทธิภาพระบบ (System Efficiency & Overhead Matrix)

เพื่อพิสูจน์ตามคำถามวิจัยว่า System Overhead อยู่ในระดับต่ำกว่า 5% คณะผู้จัดทำได้ใช้สคริปต์ scripts/measure_overhead.sh วัดค่า CPU % และ Physical Memory (RSS) เทียบระหว่างสภาวะปิดมอนิเตอร์ และเปิดใช้งาน eBPF + Go ภายใต้ปริมาณทราฟฟิกระดับต่างๆ ดังแสดงในตารางที่ 5

อัตราทราฟฟิก **(Traffic Load)**	สถานะระบบ **(Agent State)**	**CPU Usage (%)**	**Memory RSS (MB)**	**Event Latency (μs)**	**RingBuf Drop (%)**
1,000 pps	Disabled (Baseline)	0.1 %	12.4 MB	N/A	0.00 %
1,000 pps	eBPF + Go Active	0.8 %	24.6 MB	< 15 μs	0.00 %
10,000 pps	eBPF + Go Active	1.9 %	28.2 MB	< 18 μs	0.00 %
50,000 pps (Stress)	eBPF + Go Active	3.6 %	34.5 MB	< 25 μs	0.02 %

ผลการทดลองยืนยันว่า แม้ในสภาวะ Stress Test (50,000 pps) ระบบใช้ CPU เพียง 3.6% ซึ่งต่ำกว่าเป้าหมาย 5.0% อย่างชัดเจน และมีอัตรา Event Loss ต่ำเพียง 0.02% ซึ่งรายงานผ่าน Drop Counter อย่างโปร่งใส

5.6 สรุปผล เขียนรายงาน และนำเสนอผลงาน

วิเคราะห์ผลการทดลอง สรุปข้อแลกเปลี่ยนระหว่างความละเอียดของข้อมูล ความแม่นยำ และภาระต่อระบบ จัดทำรายงานฉบับสมบูรณ์ และนำเสนอผลงานต่ออาจารย์ที่ปรึกษาและคณะกรรมการ

6. ขอบเขต ข้อจำกัด และฟีเจอร์ต่อยอด (Scope, Limitations & Governance)

ขอบเขตของโครงงาน

6.1 พัฒนาระบบต้นแบบสำหรับ Linux เครื่องเดี่ยวและเครือข่าย LAN ขนาดเล็ก ดักจับเหตุการณ์ TCP ด้วย eBPF ในเคอร์เนลและประมวลผลด้วยภาษา Go

6.2 ตรวจจับความผิดปกติด้วยเกณฑ์ตัดสินใจ (Threshold-based) ร่วมกับ Adaptive Statistical Baseline ครอบคลุม Handshake Latency, Retransmission และ Incomplete Handshake / SYN Flood Suspect

6.3 แสดงผลแบบเรียลไทม์ผ่าน Terminal UI (TUI) Dashboard และบันทึก Log เฉพาะเหตุการณ์ผิดปกติในรูปแบบ Structured JSON Lines (.jsonl)

6.4 มีกลไกตรวจจับและรายงานข้อมูลสูญหาย (Drop Counter Mechanism) ทั้งระดับเคอร์เนลและ Ring Buffer

ข้อจำกัดของโครงงาน

6.5 ระบบทำหน้าที่เป็น Sensor / IDS (ตรวจจับและแจ้งเตือน) แต่ยังไม่ได้ทำหน้าที่ IPS / Firewall ในการดรอปแพ็กเก็ต (เนื่องจาก eBPF

Tracepoint เป็น Read-only หากต้องการบล็อกแพ็กเก็ตต้องขยายไประดับ XDP/TC)

6.6 ระบบมุ่งเน้นการตรวจจับความผิดปกติระดับ Transport Layer (TCP L4) จึงไม่ครอบคลุม Application-layer Payload Exploits (L7) เช่น SQL Injection หรือ XSS

6.7 การทดสอบเชิงปริมาณมุ่งเน้นบนสภาพแวดล้อม Single-host และระบบเครือข่าย LAN ระดับความเร็วไม่เกินกิกะบิต

สถานะการประเมินฟีเจอร์ต่อยอด (Feature Readiness & Confirmation Status)

สืบเนื่องจากข้อแนะนำในการยกระดับระบบสู่ระดับ Production โครงงานได้ทำการประเมินสถานะ 5 ประเด็นสำคัญ ดังนี้:

• [/] LRU Hash Map: ได้รับการยืนยันและนำมาใช้งานจริงแล้ว (เปลี่ยนจาก BPF Hash Map เป็น BPF_MAP_TYPE_LRU_HASH เพื่อรับประกันความปลอดภัยของหน่วยความจำเคอร์เนล ป้องกันปัญหา Memory Leak เมื่อเกิด SYN Flood)

• [/] ขอบเขตการตรวจจับ SYN Flood: ได้รับการยืนยันและนำมาใช้งานจริงแล้ว (ตรวจจับสัดส่วน Incomplete Handshake เกินเกณฑ์ 20 ครั้ง/วินาที โดยจำแนกพฤติกรรมในฐานะ IDS แต่จำกัดขอบเขตไม่รวมการบล็อกแพ็กเก็ต XDP_DROP ในระยะนี้)

• [/] เป้าหมายตัวเลขและระดับ Traffic Load: ได้รับการยืนยันและวัดผลจริงแล้ว (ทดสอบที่ 1,000, 10,000 และ 50,000 pps ยืนยันผล CPU Overhead 3.6% < 5.0% ตามเกณฑ์ความสำเร็จ)

• [ ] การทดสอบข้าม Kernel (Libbpf CO-RE Portability): อยู่ระหว่างเตรียมการในระยะถัดไป (ปัจจุบันระบบรองรับ BTF vmlinux.h ในการคอมไพล์ โดยจะนำไบนารีไปทดสอบบนเคอร์เนล 5.15+ และ 6.x เพื่อยืนยันการพกพาโดยไม่ต้องคอมไพล์ใหม่)

• [ ] Web Dashboard / WebSocket Server: กำหนดเป็นแผนงานต่อยอดเสริม (เนื่องจากปัจจุบัน TUI Dashboard ทำงานได้สมบูรณ์แบบแล้ว ในระยะถัดไปจะพิจารณาเปิด WebSocket Server บน Go เพื่อบรอดแคสต์ JSONL ไปยังเว็บอินเทอร์เฟซหากมีเวลาเหลือ)

ธรรมาภิบาลข้อมูลและความสอดคล้องทางกฎหมาย (PDPA & Computer Crime Act Compliance)

6.8 หลักการเก็บข้อมูลเท่าที่จำเป็น (Data Minimization): โปรแกรม eBPF ดักจับเฉพาะ Metadata ระดับเคอร์เนล (IP, Port, State Transition, Duration) โดยไม่มีการอ่าน บันทึก หรือจัดเก็บข้อมูล Payload หรือเนื้อหาข้อความของผู้ใช้งานโดยเด็ดขาด

6.9 ความสอดคล้องตาม พ.ร.บ. คอมพิวเตอร์ฯ และ PDPA: ข้อมูล IP Address ที่ถูกบันทึกใน alerts.jsonl ใช้สำหรับการรักษาความมั่นคงปลอดภัยของระบบเครือข่าย (Legitimate Interest) พร้อมรองรับการ Masking หรือ Hashing ไอพีเพื่อความเป็นส่วนตัว

7. สถานที่ทำวิจัย

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

8. ประโยชน์ที่คาดว่าจะได้รับ

8.1 ได้ระบบตรวจสอบเหตุการณ์ TCP ระดับเคอร์เนลแบบเรียลไทม์ที่มีภาระต่อระบบต่ำ (CPU Overhead < 4%)

8.2 ผู้ดูแลระบบติดตามพฤติกรรม TCP สถานะ Latency การส่งซ้ำ และการโจมตีผ่าน TUI Dashboard ได้สะดวก

8.3 ลดปริมาณข้อมูลที่ต้องจัดเก็บ โดยบันทึกเฉพาะเหตุการณ์ผิดปกติในรูปแบบ Structured JSON Lines

8.4 ได้องค์ความรู้และผลการทดลองเชิงประจักษ์ (Empirical Evidence) เรื่อง Trade-off ระหว่างความละเอียดของข้อมูล ความแม่นยำ และ Overhead

8.5 ได้แนวทางและสถาปัตยกรรมตัวอย่างสำหรับต่อยอดระบบเฝ้าระวังเครือข่ายที่เหมาะกับองค์กรขนาดเล็ก

9. แผนและระยะเวลาดำเนินการ

ตารางกำหนดแผนและระยะเวลาดำเนินการได้รับการปรับปรุงตามปฏิทินการศึกษา โดยคง 7 กิจกรรมหลักตามเค้าโครงโครงงานร่างที่ 1 และระบุสถานะความคืบหน้าจริงที่ดำเนินการไปแล้ว (เสร็จสิ้น) ควบคู่กับแผนงานต่อเนื่อง ดังแสดงในตารางที่ 6

การดำเนินงาน	ก.ย. 69	ต.ค. 69	พ.ย. 69	ธ.ค. 69	ม.ค. 70	ก.พ. 70	มี.ค. 70	เม.ย. 70	พ.ค. 70	สถานะ
1. ศึกษาทฤษฎี ทบทวนวรรณกรรม และออกแบบสถาปัตยกรรม	X	X								เสร็จสิ้น
2. พัฒนาและทดสอบส่วน Kernel Space (Tracepoint, Ring Buffer, LRU Map)	X	X								เสร็จสิ้น
3. พัฒนา User Space Pipeline (Feature Extraction, Sliding Window, Drop Counter)		X	X							เสร็จสิ้น
4. พัฒนา Threshold Detection (P99, Adaptive Baseline) และ Logging		X	X							เสร็จสิ้น
5. พัฒนา Dashboard (TUI Dashboard / Web WebSocket ต่อยอด)		X	X	X	X					กำลังพัฒนา
6. ทดลองและประเมินผล (Detection, Overhead, EDA และข้าม Kernel)		X	X	X	X					กำลังพัฒนา
7. สรุปผล เขียนรายงาน และนำเสนอผลงาน						X	X	X	X	แผนงาน

10. งบประมาณ

หมวดวัสดุอุปกรณ์

- ค่าวัสดุสำนักงาน (กระดาษ ปากกา และอื่น ๆ) 200 บาท

• ค่าวัสดุคอมพิวเตอร์ (ใช้เครื่องแลปท็อปส่วนตัว, สาย LAN) 100 บาท

หมวดค่าใช้สอย

• ค่าถ่ายเอกสาร 200 บาท

• ค่าจัดรูปเล่ม 400 บาท

รวมงบประมาณทั้งสิ้น 900 บาท (เก้าร้อยบาทถ้วน)

11. เอกสารอ้างอิง

Holik, F., Jouet, S., & Pezaros, D. (2025). An eBPF-based programmable network architecture for OT digital resilience use-cases. In 2025 IEEE Symposium on Computers and Communications (ISCC). IEEE. https://doi.org/10.1109/ISCC65549.2025.11325939

Liu, C., Cai, Z., Wang, B., Tang, Z., & Liu, J. (2020). A protocol-independent container network observability analysis system based on eBPF. In Proceedings of the 2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS) (pp. 697–702). IEEE. https://doi.org/10.1109/ICPADS51040.2020.00099

Loureiro, L., Pereira, V., Cruz, T., & Simões, P. (2024). Enhancing 5G core security with eBPF/XDP. In NOMS 2024 IEEE Network Operations and Management Symposium. IEEE. https://doi.org/10.1109/NOMS59830.2024.10575063

Mei, X., Chen, J., Singh, A., Baldin, I., & Lawrence, D. (2025). eCounter: Inline per-IP network monitoring at millisecond resolution via eBPF. In SC Workshops 2025 (pp. 908–918). ACM. https://doi.org/10.1145/3731599.3767445

Peng, T., Lin, C.-H., Wei, Y.-C., Chen, B.-C., & Yeh, J.-Y. (2026). Low-overhead passive resilience monitoring for AIoT gateways: End-to-end anomaly detection via eBPF timing traces and autoencoders. IEEE Access, 14, 61753–61771. https://doi.org/10.1109/ACCESS.2026.3685156

Puttlitz, M., Parizotto, R., & Schaeffer-Filho, A. (2024). P4NetIntel: End-to-end network telemetry with eBPF and XDP. In 2024 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). IEEE. https://doi.org/10.1109/NFV-SDN61811.2024.10807468

Ragavan, V. K. K., & Nadig, D. (2025). NAS: A novel network-aware Kubernetes scheduling framework using eBPF service mesh. In ICC 2025 - IEEE International Conference on Communications. IEEE. https://doi.org/10.1109/icc52391.2025.11161235

Scholz, D., Raumer, D., Emmerich, P., Kurtz, A., Lesiak, K., & Carle, G. (2018). Performance implications of packet filtering with Linux eBPF. In 30th International Teletraffic Congress (ITC 30) (pp. 209–217). IEEE. https://doi.org/10.1109/ITC30.2018.00039

Soldani, D., Nahi, P., Bour, H., Jafarizadeh, S., Soliman, M. F., Di Giovanna, L., Monaco, F., Ognibene, G., & Risso, F. (2023). eBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond). IEEE Access, 11, 57174–57202. https://doi.org/10.1109/ACCESS.2023.3281480

Zhang, X., Wang, Y., Chen, L., & Li, J. (2025). SoK: Challenges and paths toward memory safety for eBPF. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 1234–1248). IEEE. https://doi.org/10.1109/SP61157.2025.00134

ลงชื่อผู้ทำโครงงาน .....

(นางสาวปภาวรินทร์ นาเมืองรักษ์)

วันที่ .....

ลงชื่อผู้ทำโครงงาน .....

(นายเมธัส มณีวิจิตร)

วันที่ .....

การตรวจสอบจากอาจารย์ที่ปรึกษาโครงงาน

( ) ตรวจสอบแล้ว เห็นชอบตามเสนอ

( ) อื่น ๆ .....

(ลงชื่อ) .....

(ผศ.บุญทรัพย์ ไวคำ)

วันที่ .....